

**Vivere: Design and Development of a Personalized Fashion Recommendation and
Discovery Web Application.**

Jasmine A. Wagieh

2200250

Computer Science

Coventry University at The Knowledge Hub

KH6003CMD: Dissertation and Project Artefact

Dr. Marwa Refai, Dr Nada Gaballah

12th of August 2026

Github: <https://github.com/jasmine-wagieh/Vivere-Project>

Abstract

Vivere is a full-stack fashion discovery and recommendation web application designed to provide users with a more personalized and interactive approach to exploring clothing and fashion products. The system was developed to address the limitations of traditional fashion catalogues, where users are often required to manually browse large numbers of products with limited personalization. Vivere combines product discovery, user preferences, recommendation functionality and user-generated content within a single platform.

The application was developed using React for the frontend and Node.js with Express for the backend, while MongoDB Atlas was used for persistent data storage. The system contains a fashion dataset of approximately 44,000 items and provides search, filtering, pagination and detailed product information. Registered users can maintain a Wishlist, upload their own fashion items and receive personalized recommendations. A Personal Stylist feature allows users to specify preferences such as gender, occasion, season, colour and product type to generate relevant fashion suggestions. The recommendation system also considers previously liked items to develop a basic representation of the user's fashion preferences.

Additional discovery functionality includes finding similar products within the Vivere dataset and a Shop Similar feature that enables users to search external online stores for comparable products. Uploaded items are integrated into the application's searchable catalogue, allowing user-generated content to participate in the wider discovery experience.

The completed system demonstrates how full-stack web technologies and recommendation techniques can be combined to create an accessible and personalized fashion discovery platform.

1.Introduction

The fashion industry has increasingly adopted digital technologies to change how consumers discover, evaluate and purchase clothing. Online fashion platforms provide access to extremely large product catalogues; however, the quantity of available products can also make finding relevant items difficult. Users may need to navigate through hundreds or thousands of products before finding clothing that matches their individual preferences. Search and filtering can reduce this problem, but conventional approaches often treat every user in the same way and provide limited consideration of individual tastes and previous interactions.

Vivere was developed as a personalized fashion discovery web application designed to make exploring fashion products more efficient and user-centered. Rather than functioning only as a conventional online catalogue, Vivere combines several methods of product discovery within one system. Users can browse a large fashion dataset, search for specific products, apply filters, save items to a Wishlist, discover similar products and receive personalized recommendations. The application also includes a Personal Stylist feature through which users can explicitly specify fashion preferences and receive a collection of products matching those requirements.

A central motivation behind the project was to investigate how personalization could improve the experience of navigating a large fashion dataset. Vivere uses a dataset containing approximately 44,000 fashion products with attributes including gender, category, article type, color, season and usage. These attributes provide the foundation for the application's search, filtering and recommendation functionality. Instead of presenting the dataset as a static collection, Vivere uses this information to connect products to user preferences and interactions.

Personalization within Vivere occurs through both explicit and implicit information. Explicit preferences are collected through the Personal Stylist, where users can select characteristics such as gender, occasion, preferred season, favorite colors and the types of products they want to discover. Implicit preferences are obtained from products that users add to their Wishlist. These interactions can be analyzed to identify recurring characteristics within products that a user has previously liked and subsequently influence the recommendations presented to that user. This creates a more personalized experience than displaying identical recommendations to every user.

Another important objective of the project was to support different methods of fashion discovery. A user may find an interesting item without knowing the correct terminology required to search for related products. Vivere therefore includes a Find Similar feature that

identifies related items from the application's dataset based on their product characteristics. A Shop Similar feature extends this discovery process beyond Vivere by allowing users to search external online shopping platforms for comparable products.

The system also supports user-generated fashion content. Registered users can upload clothing items together with information describing the product. Uploaded images and associated product information are stored by the system, and the resulting items are integrated into the searchable fashion catalogue. This allows Vivere to combine the original dataset with content contributed by its users rather than limiting the application to a fixed collection of products.

From a technical perspective, Vivere was implemented as a full-stack web application. React was used to develop the interactive frontend interface, while Node.js and Express were used to implement the backend and REST API functionality. MongoDB Atlas provides persistent database storage for application and user data. The architecture separates the user interface, server-side processing and database responsibilities, allowing the different components of the application to communicate through defined API endpoints.

1.1 Project Aim

The primary aim of Vivere is to design and develop a full-stack fashion discovery application that improves the process of finding relevant fashion products through search, personalization, recommendations and interactive discovery features.

1.2 Project Objectives

To achieve this aim, the project established the following objectives:

1. Develop an accessible web interface for browsing a large fashion product dataset.
2. Implement user registration and authentication to support personalized functionality.
3. Provide product search, filtering and pagination capabilities.
4. Allow registered users to save preferred products through a Wishlist system.
5. Develop a Personal Stylist that generates recommendations from explicitly selected user preferences.
6. Generate personalized recommendations using information obtained from users' liked products.

7. Implement functionality for identifying similar products within the Vivere catalogue.
8. Enable users to upload their own fashion products and images.
9. Integrate uploaded products into the application's searchable catalogue.
10. Provide a Shop Similar function that connects product discovery with external online shopping searches.
11. Store and manage application data using a persistent cloud-hosted database.
12. Test and evaluate the completed system to determine whether the functional requirements and project objectives have been achieved.

1.3 Report Structure

The remainder of this report describes the research, design, development and evaluation of Vivere. It first considers the background of fashion recommendation and product discovery systems before presenting the requirements and design decisions used for the application. The system architecture, database structure and major implementation components are then discussed, followed by testing and evaluation of the completed system. Finally, the report examines the limitations of the current implementation and identifies potential areas for future development.

. Background and Literature Review

2.1 Digital Fashion and Product Discovery

The growth of e-commerce has significantly changed the way consumers discover fashion products. Online platforms allow retailers to provide catalogues containing thousands of products, giving consumers considerably more choice than would normally be available within a physical store. However, the increased number of available products creates a different problem: users must identify relevant items within very large collections. Recent fashion-industry research identifies product discovery as an important challenge, with consumers increasingly expecting digital platforms to help them locate relevant products rather than simply presenting large catalogues. McKinsey's *State of Fashion 2025* specifically identifies the difficulty consumers experience when attempting to find products within extensive online selections and highlights AI-supported curation, search and recommendation as potential approaches to improving product discovery.

This issue is directly relevant to Vivere. The dataset used within the application contains approximately 44,000 fashion items, meaning that displaying products without

mechanisms for narrowing the collection would create a poor discovery experience. Consequently, Vivere combines conventional information-retrieval mechanisms, such as keyword search and attribute filtering, with recommendation and similarity functionality.

Search and recommendation serve related but different purposes. Search is particularly effective when users already understand what they are looking for and can express this through keywords or filters. Recommendation is useful when the user has less clearly defined requirements or when the system has information that can be used to predict relevant products. Vivere therefore incorporates both approaches rather than treating recommendation as a replacement for conventional search.

2.2 Recommender Systems

Recommender systems are software mechanisms designed to identify items that may be relevant to individual users. They have become important because digital services frequently contain considerably more information or products than a user can practically examine. Bobadilla et al. (2013) describe several established recommendation approaches, including demographic, content-based and collaborative filtering techniques.

A major distinction exists between **content-based filtering** and **collaborative filtering**. Content-based recommendation uses characteristics associated with individual items and attempts to recommend products containing characteristics that correspond with the user's interests. Collaborative filtering instead considers patterns of interaction across multiple users. For example, if several users demonstrate similar behaviour, products preferred by one user may be recommended to another.

Collaborative filtering has been successfully applied across many recommendation domains; however, it introduces challenges including data sparsity and the cold-start problem. Data sparsity occurs when there are insufficient interactions between users and products to establish reliable patterns, while cold start occurs when new users or items have little or no interaction history. These remain significant challenges within contemporary recommender-system research.

For Vivere, these considerations influenced the decision to use product attributes and individual user interactions rather than implementing a fully collaborative recommendation system. A university prototype may contain relatively few active users compared with a commercial platform, making collaborative filtering considerably less useful because insufficient user interaction data would exist. In contrast, the fashion dataset already contains descriptive information for each item, including article type, category, color, gender, season and usage. These attributes can immediately contribute to

recommendation decisions without requiring thousands of users to interact with the application first.

2.3 Fashion Recommendation Systems

Fashion recommendation presents additional challenges compared with recommendations in some other domains. Fashion preferences can depend on color, garment type, style, season, occasion, compatibility with other garments and individual aesthetic preferences. Consequently, recommending clothing involves more than identifying products that belong to the same general category.

Deldjoo et al. (2022) identify fashion recommendation as a specialized recommendation domain and distinguish between tasks such as individual item recommendation, outfit recommendation, size recommendation and compatibility prediction. Their review also emphasizes the importance of different types of side information relating to users, products and context when constructing fashion recommendation systems. Similarly, Ding et al. (2023) describe personalized fashion-product recommendation, complementary recommendation and outfit recommendation as distinct computational fashion tasks and highlight the gap that can exist between academic recommendation techniques and practical requirements within the fashion industry.

Vivere primarily focuses on **personalized fashion-product recommendation and discovery** rather than outfit generation or size prediction. The purpose of the application is to identify individual fashion products that correspond with a user's preferences and provide multiple pathways through which users can discover related items.

The characteristics contained within Vivere's dataset are particularly appropriate for this approach. For example, an item may contain attributes identifying it as women's apparel, a dress, red, suitable for summer and intended for casual use. These characteristics can be compared against preferences explicitly selected by the user or characteristics repeatedly appearing among products that the user has liked.

2.4 Explicit and Implicit User Preferences

Personalized systems require some representation of user preferences. This information can be obtained explicitly by asking users what they prefer or implicitly by observing their interactions with the system.

Vivere uses both approaches. The Personal Stylist represents explicit preference collection. Users can specify characteristics such as the gender they are shopping for, occasion, preferred season, favorite colors and product types. The application then searches the fashion collection for products corresponding with those selections.

The Wishlist provides an additional source of implicit preference information. Adding an item to a Wishlist represents a stronger indication of interest than simply viewing an item. Vivere can therefore examine characteristics shared between liked products and construct a basic taste profile. For example, if a user's Wishlist repeatedly contains blue products, summer products and particular article types, those characteristics can contribute to subsequent recommendations.

The use of implicit behavior is established within recommender-system research. Modern recommender systems may use behavioral information alongside explicit ratings or preferences to infer what users are interested in. This approach is particularly appropriate for Vivere because requiring users to repeatedly rate products would increase interaction effort. Wishlist behavior instead provides useful information through an action that already has a practical purpose within the application.

2.5 Personalization in Online Fashion

Personalization is becoming increasingly important within digital retail. Industry research suggests that consumers expect online systems to reduce the effort involved in discovering relevant products. McKinsey reports that 82% of surveyed consumers wanted AI to reduce the amount of time required to research purchases, while 79% indicated that it would be helpful for AI to understand their specific requirements and recommend products. Product discovery and customer search was also the highest-ranked generative-AI use case among surveyed fashion executives for 2025.

Although Vivere does not attempt to reproduce the complex artificial-intelligence infrastructure used by major commercial retailers, these developments demonstrate the relevance of the project's central problem. The Personal Stylist and recommendation engine represent a smaller-scale implementation of the same general principle: reducing the amount of manual searching required by using available information about the user to narrow a large product catalogue.

The design also intentionally maintains user control. Recommendations complement rather than replace browsing, searching and filtering. A user who does not wish to use personalized functionality can continue exploring the catalogue manually.

2.6 Similar-Item Discovery

Another important approach to product discovery is similarity-based navigation. Instead of requiring the user to formulate a new search query, the application can use an existing item as the starting point for further discovery.

Vivere's **Find Similar** functionality follows this principle. Product metadata is used to locate other products with related characteristics within the Vivere dataset. This is especially useful when a user finds an appealing product and wants alternatives but does not know exactly which search terms would reproduce its characteristics.

The **Shop Similar** feature extends this concept beyond the internal catalogue. Rather than limiting discovery to products stored by Vivere, the application constructs relevant searches that allow users to continue looking for comparable products through external online shopping services. This feature does not constitute a direct retailer inventory API; instead, it acts as a bridge between the application's internal fashion-discovery process and external product searches.

2.7 User-Generated Content

Vivere also introduces user-generated content through its upload functionality. A registered user can upload a fashion image and provide descriptive information for the item. The image and associated metadata are then incorporated into the application so that the uploaded product can appear within subsequent searches.

This functionality changes the system from a completely static dataset interface into an extensible platform. The original dataset provides the initial collection, while user contributions can expand the catalogue over time. The same product structure used by the existing dataset is also used to describe uploaded items, allowing them to participate in existing discovery mechanisms.

2.8 Summary

Existing research demonstrates that recommender systems can reduce information overload by selecting products according to user interests, while fashion recommendation introduces additional requirements associated with product attributes, context and aesthetic preferences. Content-based approaches are particularly suitable when rich item metadata is available and interaction data across large numbers of users is limited.

These findings influenced the design of Vivere. The application combines traditional search and filtering with explicit preference selection, Wishlist-derived personalization, similar product discovery and user-generated content. Rather than implementing a highly complex machine-learning model, the project prioritizes an interpretable recommendation approach appropriate to the available dataset and scale of the application. This provides the foundation for the system requirements and design discussed in the following section.

3. Analysis and System Design Requirements

3.1 Requirements Analysis

The requirements for Vivere were defined around the central objective of improving fashion discovery through personalization, recommendation and user interaction. The system was required to support both conventional browsing and more intelligent discovery mechanisms so that users could choose how they wanted to explore the fashion catalogue.

The functional requirements were derived from the main activities a user should be able to complete within the system. These include account creation, authentication, browsing, searching, filtering, saving products, receiving recommendations, uploading new products and following links to external shopping services.

The principal functional requirements were:

1. The system must allow users to register for an account and log in.
2. The system must display fashion products from the dataset.
3. Users must be able to search for products using keywords.
4. Users must be able to filter products by characteristics such as gender, colour, season and usage.
5. Users must be able to view detailed information about individual products.
6. Logged-in users must be able to like and unlike products.
7. Liked products must be stored in a wishlist associated with the user.
8. The system must provide similar-product recommendations from the internal catalogue.
9. The system must allow users to specify style preferences through the Personal Stylist.
10. The system must generate personalized recommendations from those preferences.
11. The system must generate recommendations based on characteristics of products previously liked by the user.
12. Users must be able to upload fashion products with an image and descriptive information.
13. Uploaded products must be stored persistently and become available within the searchable catalogue.

14. The application must provide a Shop Similar feature that directs users to external online searches for comparable products.
15. The application must preserve relevant user information between sessions where appropriate.

These functional requirements were prioritized because they represent the main user journeys supported by Vivere. Features such as search and filtering provide predictable control, while the recommendation features support more exploratory forms of product discovery.

3.2 Non-Functional Requirements

In addition to functional requirements, several non-functional requirements influenced the implementation of the application.

Usability was an important consideration because fashion discovery should require minimal technical knowledge. Navigation controls were therefore kept visible and major features such as Personal Stylist, Upload, For You and Wishlist were accessible directly from the interface.

Performance was also relevant because the fashion dataset contains approximately 44,000 products. Loading the entire dataset into the browser would create unnecessary delays and memory usage. Pagination was therefore implemented so that only a limited number of records are retrieved for each page.

Reliability required the application to handle unavailable images, invalid searches and unsuccessful requests without terminating the interface. Image fallback behavior and error messages were implemented to provide more graceful handling of these situations.

Scalability influenced the separation between frontend, backend and database functionality. A REST-style API allows the interface to request only the data it requires, while MongoDB Atlas provides cloud-hosted persistent storage that can support larger amounts of data than client-side storage.

Maintainability was considered through the division of functionality into React components and backend routes. Components such as the authentication interface, upload page and online-shopping modal separate specialized behaviors from the main application logic.

Security was relevant to account and database functionality. Environment variables were used to avoid storing the MongoDB connection string directly in source code. However, the current authentication implementation remains a prototype and therefore represents an

area requiring further improvement, particularly in relation to password hashing and production-grade session management.

3.3 Use Case Design

The use case diagram below summarizes the primary interactions between the user and Vivere

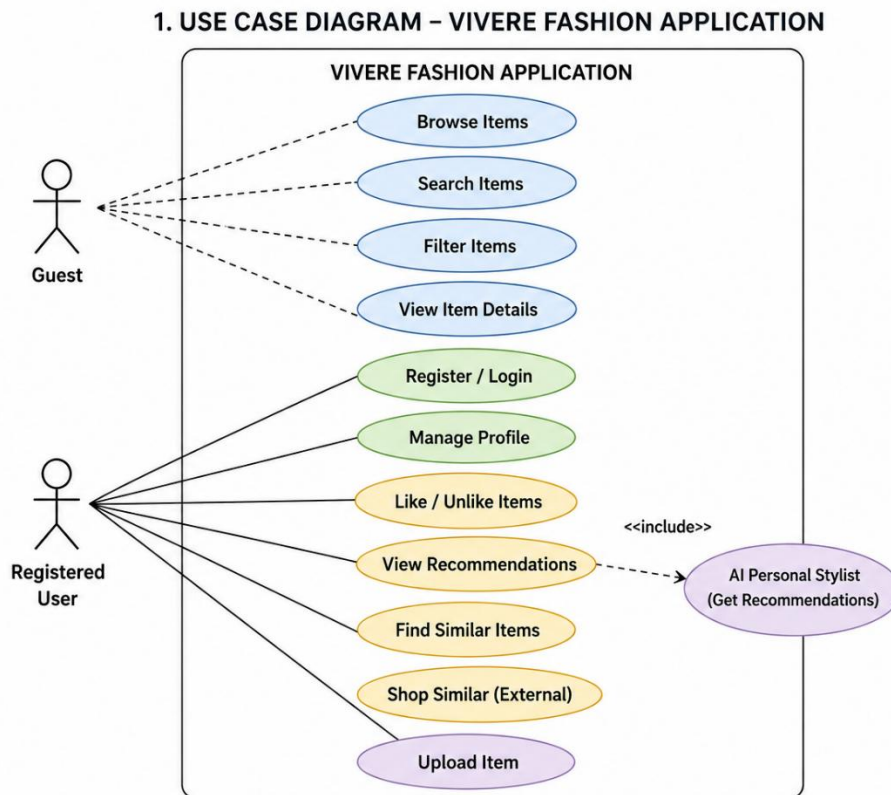


Figure 1: Vivere Use Case Diagram

The primary actor is the user. Unauthenticated users can browse products and access general discovery functionality, while authentication enables features that require persistent user-specific information such as wishlists, personalised recommendations and uploads.

The authentication use cases include registration and login. Once authenticated, the user can like products and access the wishlist, where selected product identifiers are associated with the corresponding user record.

The browsing use cases include viewing the product catalogue, searching by keyword, applying filters and opening individual product details. These features represent the conventional fashion-discovery path and do not depend on the recommendation engine.

The recommendation-related use cases introduce more personalised interactions. The Personal Stylist allows users to provide preferences explicitly, while the For You feature uses characteristics derived from liked products. Find Similar identifies comparable products within the Vivere dataset. These different mechanisms are intentionally separate because they solve different discovery problems.

The Upload Clothing use case allows the user to contribute fashion items to the catalogue by providing an image and product metadata. The uploaded product can then participate in ordinary search and discovery processes.

Finally, Shop Similar extends the user journey beyond Vivere. Rather than purchasing products directly within the application, the user can search for similar products through external online shopping services.

3.4 System Architecture

Vivere follows a client-server architecture consisting of three main layers: the React frontend, the Node.js/Express backend and the MongoDB Atlas database.

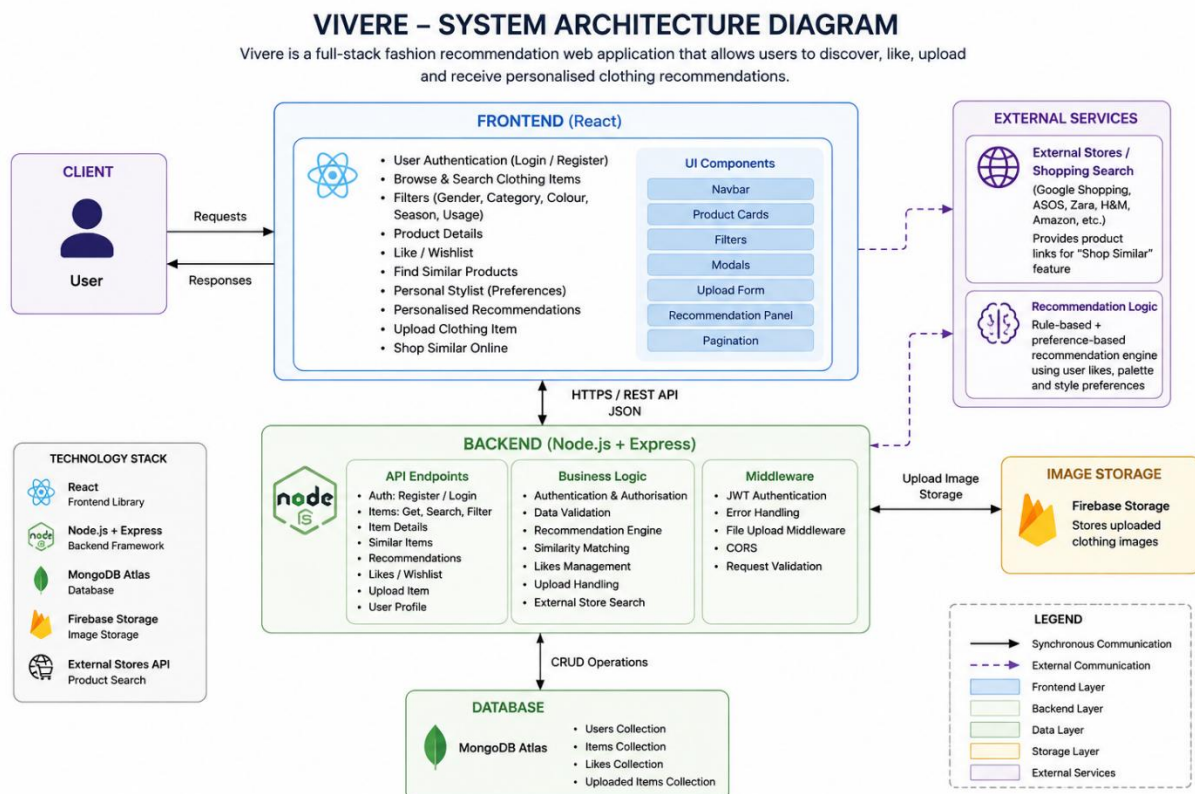


Figure 2: Vivere System Architecture Diagram

The frontend is responsible for presentation and user interaction. React manages application state and controls which content is displayed, including the shop, wishlist, Personal Stylist, recommendation view and upload interface. User actions such as performing a search or liking a product trigger HTTP requests to the backend.

The backend provides the application's server-side logic. Node.js and Express expose API routes used by the React frontend. These routes perform tasks including retrieving products, processing filters, handling authentication, updating likes, generating recommendations and storing uploaded item information.

MongoDB Atlas provides persistent storage. Product records are stored within the fashion collection, while user records contain account information and identifiers for liked products. The backend communicates with MongoDB through Mongoose, which provides schema definitions and database query functionality.

The architecture separates concerns between the interface, processing logic and data-storage layers. This structure simplifies development and allows individual parts of the application to be modified without requiring the complete system to be rewritten.

3.5 Data Model

The central product representation used within Vivere includes fields such as: id, gender, masterCategory, subcategory, articleType, baseColour, season, year, usage and productDisplayName. These attributes originate from the fashion dataset and provide structured information that can be used for browsing, filtering and recommendation.

Uploaded products extend this structure with fields such as: imageUrl, uploadedBy and isUserUpload. The imageUrl allows the application to distinguish uploaded image files from the original static dataset images. uploadedBy associates the item with the contributing user, while isUserUpload identifies records that were created through the upload interface.

The user data model contains core account information and a likedItems array. Each entry represents the numeric identifier of a product liked by the user. This design allows the backend to retrieve the corresponding products when loading the wishlist or constructing the user's taste profile.

VIVERE – DATABASE / ER DIAGRAM (MongoDB Atlas Collections)

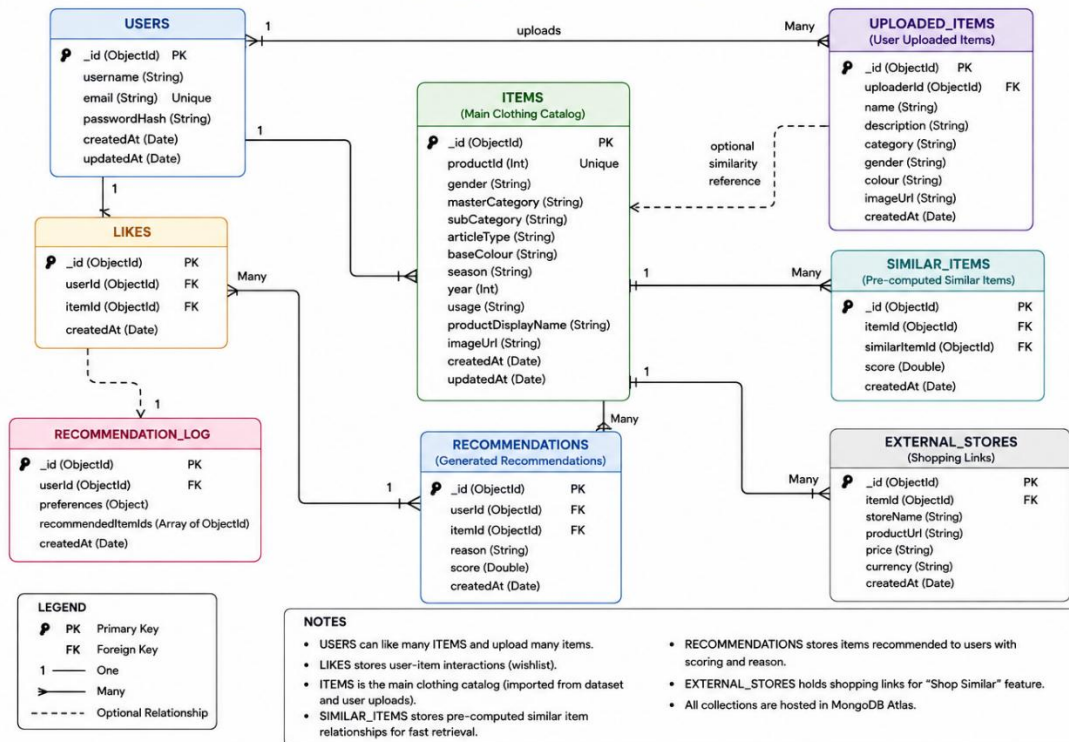


Figure 3: Vivere Database / ER Diagram

Although MongoDB is document-oriented rather than a traditional relational database, the ER representation is useful for illustrating the logical relationships between users, products, likes, uploads and recommendation functionality.

3.6 Recommendation Design

Vivere implements a content-oriented recommendation approach based on product metadata. The system does not attempt to train a complex machine-learning model. Instead, product characteristics are assigned different levels of importance according to the recommendation context.

The Personal Stylist uses explicit preferences supplied by the user. Selected colours, article types, season, usage and gender are compared against product metadata. Matching products receive scores according to the number and significance of matched characteristics, after which higher-scoring products are presented to the user.

The For You recommendation process differs because it derives preferences from items already liked by the user. The backend retrieves those products and counts repeated

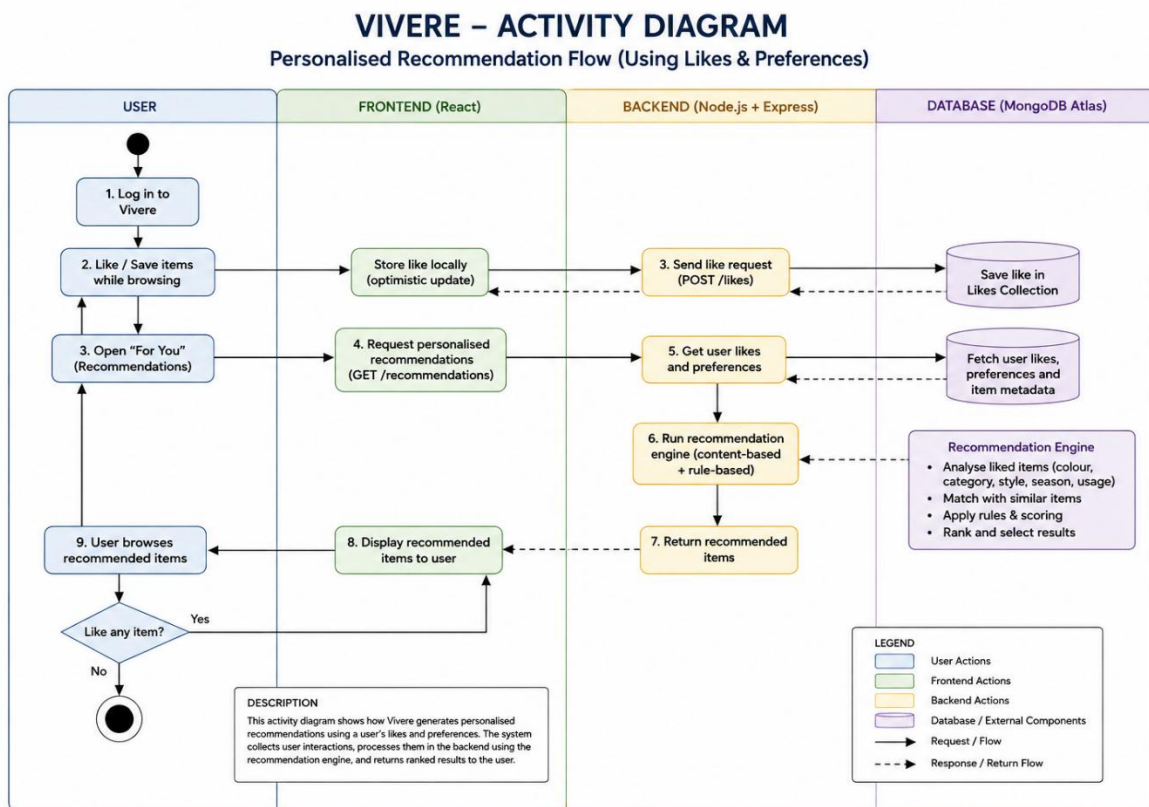
characteristics such as article type, category, colour, usage, season and gender. These counts are used to create a lightweight taste profile.

Candidate products that the user has not already liked are then scored according to how closely they match that profile. Article type and category matches receive relatively strong weighting because they describe the type of fashion product, while colour, usage, season and gender contribute additional signals.

This approach is intentionally interpretable. The application can identify which attributes contributed to recommendations, making the recommendation process easier to understand and evaluate than a completely opaque model.

3.7 Activity Flow

The recommendation process is illustrated by the activity diagram.



Although MongoDB is document-oriented rather than a traditional relational database, the ER representation is useful for illustrating the logical relationships between users, products, likes, uploads and recommendation functionality.

3.6 Recommendation Design

Vivere implements a content-oriented recommendation approach based on product metadata. The system does not attempt to train a complex machine-learning model. Instead, product characteristics are assigned different levels of importance according to the recommendation context.

The Personal Stylist uses explicit preferences supplied by the user. Selected colors, article types, season, usage and gender are compared against product metadata. Matching products receive scores according to the number and significance of matched characteristics, after which higher-scoring products are presented to the user.

The For You recommendation process differs because it derives preferences from items already liked by the user. The backend retrieves those products and counts repeated characteristics such as article type, category, color, usage, season and gender. These counts are used to create a lightweight taste profile.

Candidate products that the user has not already liked are then scored according to how closely they match that profile. Article type and category matches receive relatively strong weighting because they describe the type of fashion product, while color, usage, season and gender contribute additional signals.

This approach is intentionally interpretable. The application can identify which attributes contributed to recommendations, making the recommendation process easier to understand and evaluate than a completely opaque model.

3.7 Activity Flow

The recommendation process is illustrated by the activity diagram.

VIVERE – ACTIVITY DIAGRAM



Figure 4: Personalized Recommendation Activity Diagram

The process begins when the user interacts with products or provides explicit preferences. The frontend sends the relevant information to the backend, where recommendation logic processes the request. Product data is obtained from MongoDB, candidate products are evaluated and ranked results are returned to the React interface.

The user can then continue interacting with recommended products by viewing details, liking them or searching for similar items. These new interactions may later influence future personalized recommendations.

3.8 Upload Sequence

The upload feature introduces a different interaction because both file data and textual metadata must be transferred to the backend.

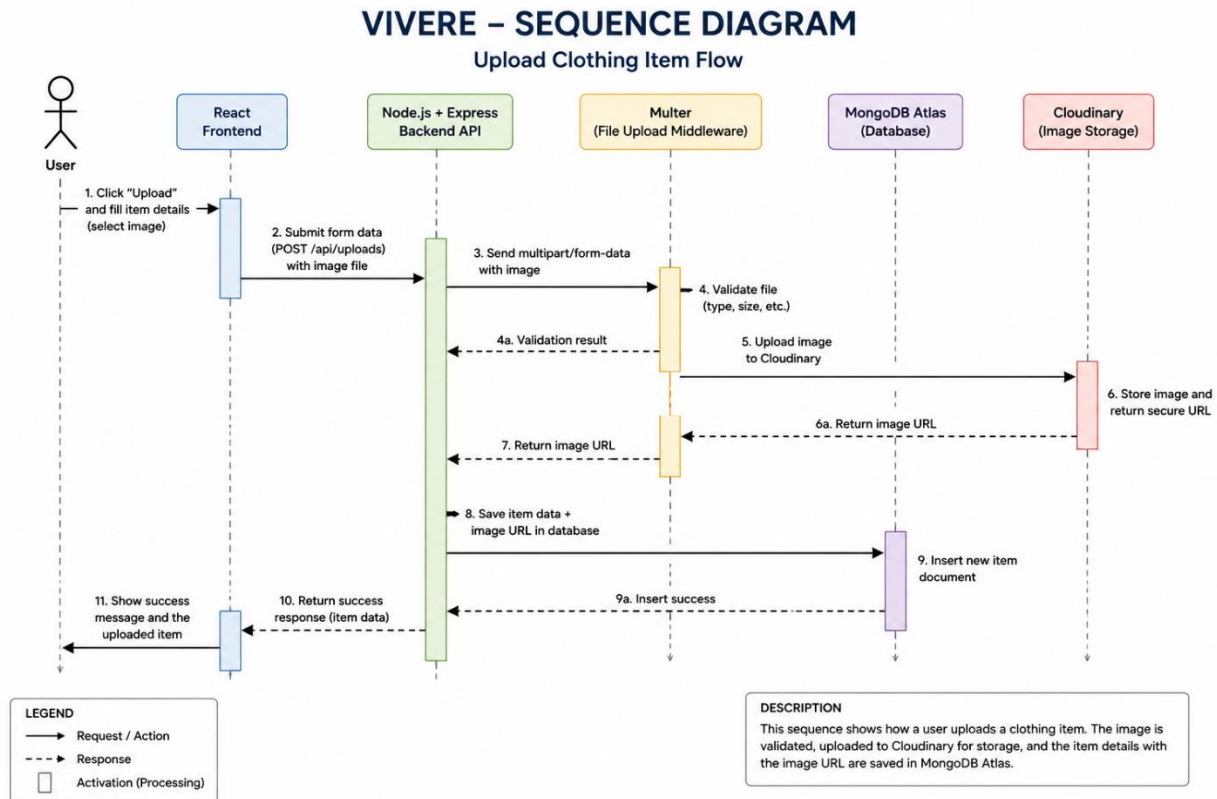


Figure 5: Upload Clothing Item Sequence Diagram

The user first selects an image and provides descriptive information through the React upload form. The form submits multipart data to the Express backend. Multer processes the uploaded file and stores it in the configured upload directory.

The backend then constructs the corresponding product record, including the generated image path and the supplied metadata. This record is written to MongoDB Atlas. After successful storage, the newly created product is returned to the frontend.

The application can then treat the uploaded product similarly to records from the original dataset. When displayed, the interface uses the stored upload path instead of attempting to load the image from the static dataset directory.

3.9 Design Summary

The design of Vivere combines conventional web application architecture with specialized fashion-discovery functionality. The separation of frontend, backend and database responsibilities supports maintainability, while the shared product structure allows original

dataset items and user-uploaded products to participate in the same discovery environment.

The next section discusses how these designs were translated into actual implementation, including the technologies, API endpoints, frontend components, recommendation logic and upload functionality.

4. Implementation and Development

4.1 Development Environment and Technology Stack

Vivere was implemented as a full-stack web application using JavaScript technologies across both the frontend and backend. The frontend was developed using **React**, while the backend was implemented using **Node.js** and **Express**. **MongoDB Atlas** was selected for cloud-hosted database storage, and **Mongoose** was used to define schemas and interact with MongoDB from the backend.

React was selected because the application contains multiple interactive views and requires frequent state updates. Examples include switching between the main shop, Wishlist, Personal Stylist, For You recommendations and upload interface. React's component-based structure made it possible to separate specialized functionalities such as authentication, uploading and external shopping search into individual components.

Node.js and Express were used to create the server-side API. Express provided the routing structure used by the frontend to retrieve products, authenticate users, update likes, generate recommendations and process uploads. The frontend communicates with the backend through HTTP requests using the Fetch API.

The primary technologies used within the project were therefore:

- React for the user interface.
- JavaScript for frontend and backend development.
- Node.js for server-side execution.
- Express for REST-style API routing.
- MongoDB Atlas for persistent cloud-hosted data.
- Mongoose for MongoDB schemas and queries.
- Multer for processing image uploads.
- HTML and CSS for interface layout and presentation.

- Git and GitHub for source-code version control.

The application was developed and tested locally, with the React development server operating on port 3000 and the Express backend operating on port 5000.

4.2 Fashion Dataset Integration

The main catalogue is based on a fashion dataset containing approximately 44,000 products. Each product contains structured metadata describing its characteristics. Important fields include id, gender, masterCategory, subCategory, articleType, baseColour, season, year, usage and productDisplayName.

A Mongoose schema was created to represent these items in MongoDB. The existing records are stored within the blueprints collection. The structured fields are particularly important because they allow the same dataset to support multiple application features rather than simply displaying static product cards.

For example, productDisplayName, articleType, subCategory and masterCategory can be searched using keywords, while fields such as gender, baseColour, season and usage support filtering. These same attributes are also reused by the recommendation system to determine whether products correspond with the user's preferences.

The backend exposes the catalogue through the /api/items route. The route accepts parameters including page number, result limit, search term, gender, category, colour, season and usage. A MongoDB filter is dynamically constructed from the selected values.

Pagination was included because loading all 44,000 products at once would create unnecessary network traffic and negatively affect frontend performance. The default page size is twelve products. The backend calculates the number of available pages from the total number of matching documents and returns both the products and pagination information to the frontend.

Search functionality uses case-insensitive regular expressions. Instead of limiting keyword search to only the product title, the search term is compared against several fields including product name, article type, category, colour, gender, usage and season. This allows users to enter queries such as "black shoes" or "summer dresses" and receive relevant results even when those exact phrases do not occur within a single field.

4.3 Frontend Product Browsing

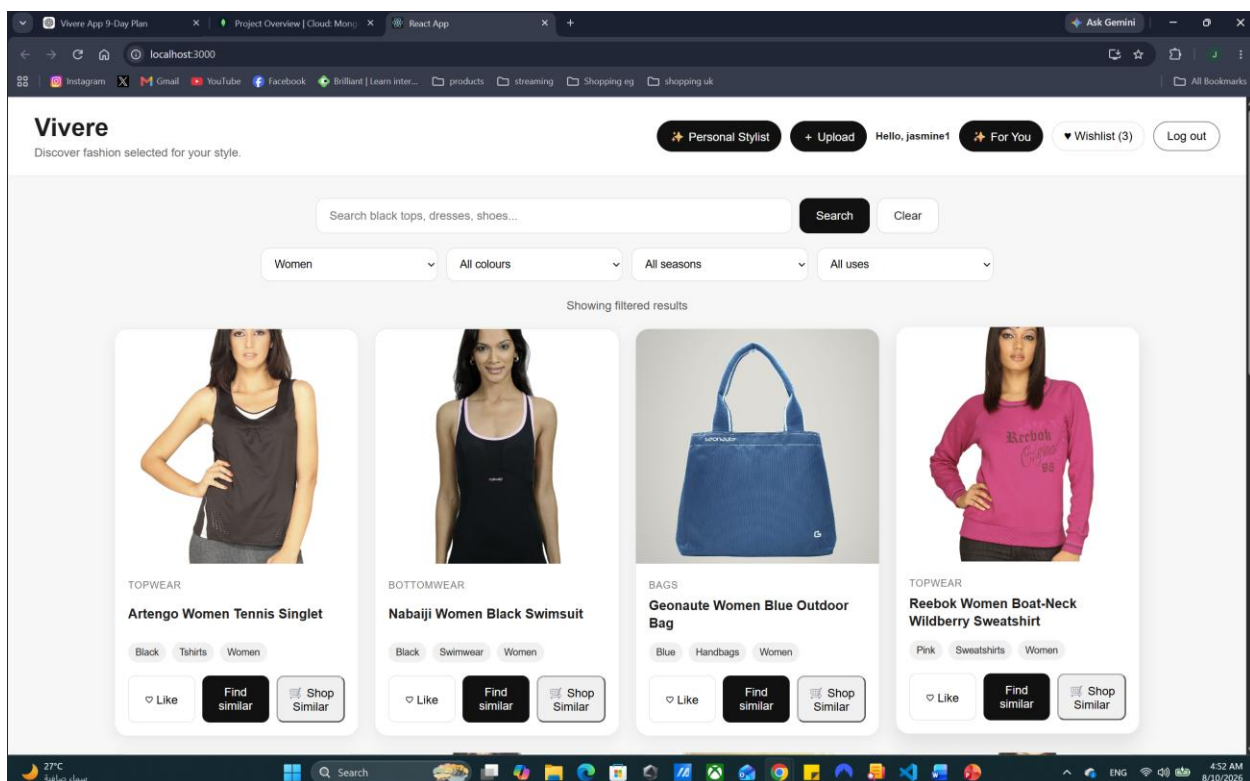
The main React application maintains state for the current collection of items, active page, total page count, loading status, search term and selected filters. Whenever one of these

values changes, a React useEffect function sends a new request to the backend and updates the displayed collection.

Each product is displayed using a reusable product-card structure. The card shows the product image, subcategory, display name, color, article type and gender. Actions are also provided for liking the product and finding similar products.

Product images from the original dataset are retrieved using the numeric product identifier. When an image is unavailable, the interface replaces it with a fallback placeholder rather than leaving a broken image element. This was necessary because some records within large datasets may refer to unavailable image files.

Clicking a product opens a details interface containing additional attributes such as season, usage and year. Separating summary cards from detailed information helps keep the main catalogue visually manageable while still allowing users to inspect individual products.



4.4 User Registration and Authentication

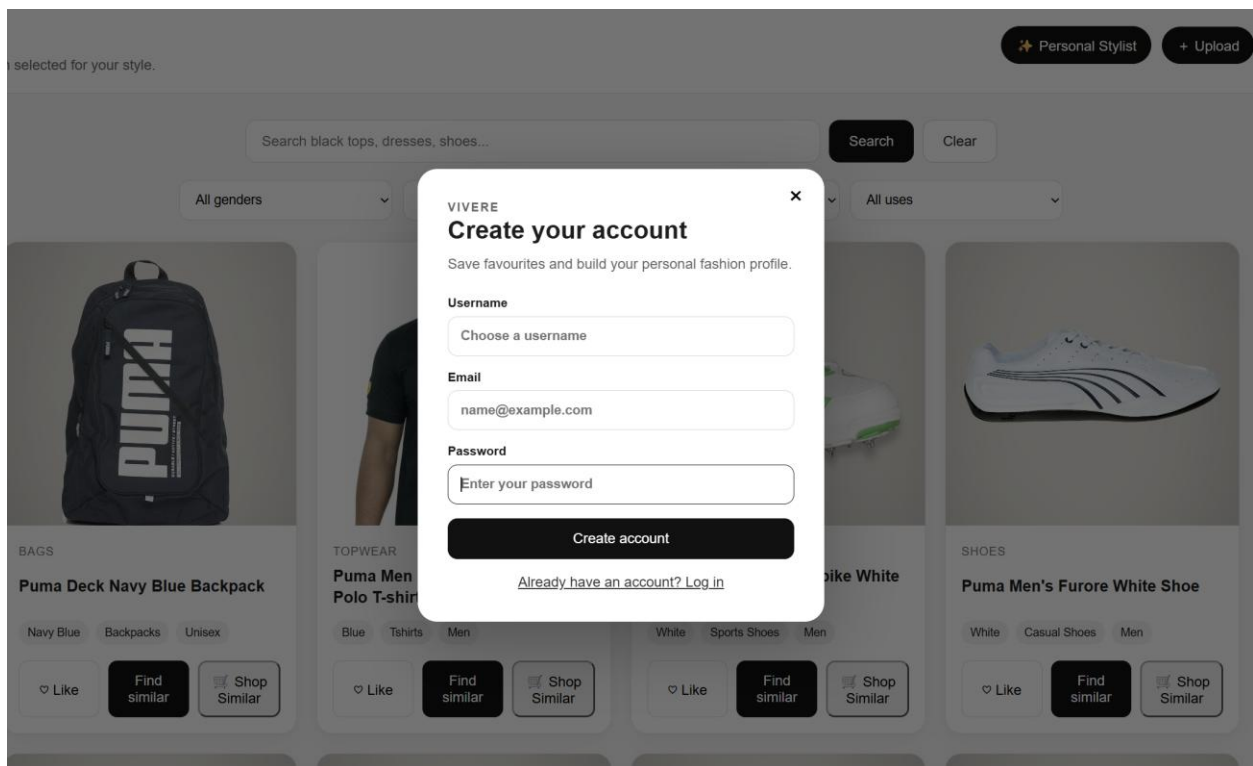
User-specific functionality requires registration and login. A separate User model was created with fields for username, email, password and likedItems.

The registration route validates that a username, email and password have been supplied. Usernames and email addresses are normalized before storage, and the backend checks for duplicate records before creating a new account.

The login route accepts either username or email together with a password. If a matching user record is found, the backend returns the user's identifier, username, email and liked-item list.

On the frontend, successful login information is stored in localStorage under the key `vivereUser`. This means the user can refresh the browser without immediately losing the logged-in interface state. The stored object is loaded when the React application starts.

The current authentication system is sufficient for demonstrating account-dependent functionality in the prototype; however, passwords are not currently protected using production-grade hashing. This limitation is addressed later in the evaluation and future-work sections.



4.5 Wishlist Implementation

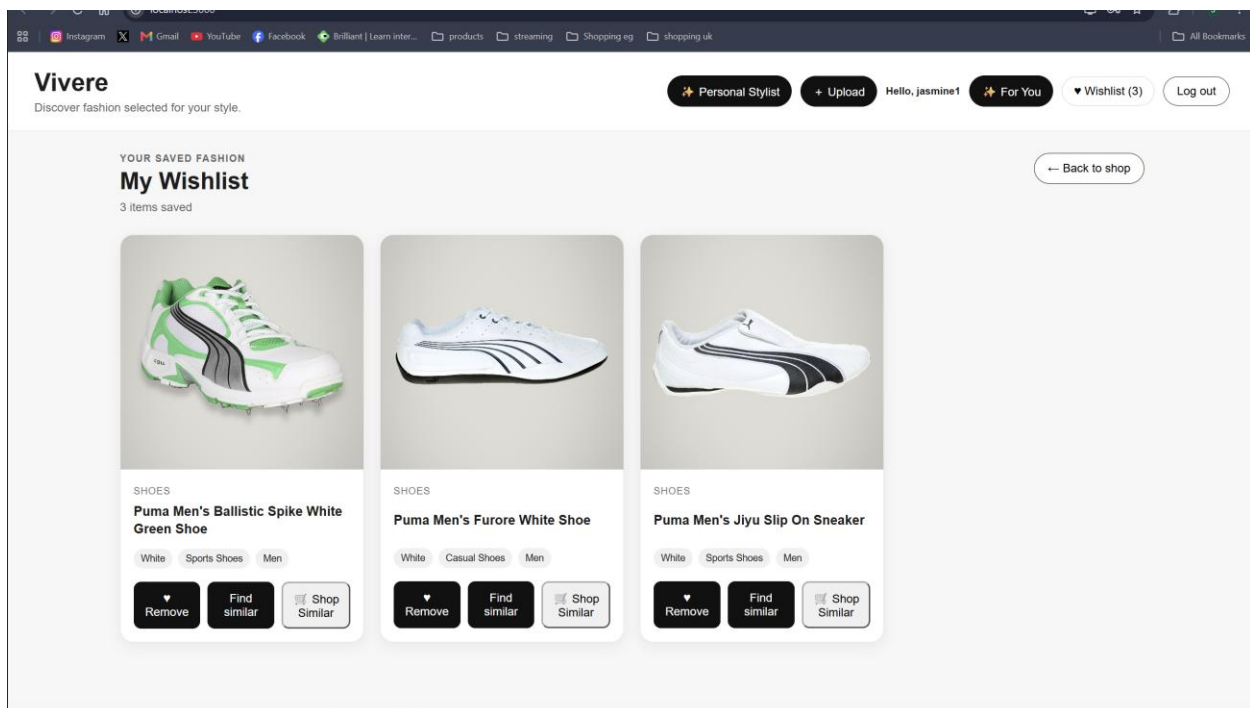
The Wishlist is implemented through the `likedItems` array associated with each user. Each value in this array represents the numeric identifier of a product.

When a user clicks the Like button, the frontend sends a POST request containing the user identifier and product identifier. The backend checks whether the item already exists within likedItems. If the item is already liked, it is removed; otherwise, it is added.

This creates a toggle behavior from a single API route. The updated liked-item list is returned to the frontend, which updates both React state and the user information stored in localStorage.

A separate Wishlist endpoint retrieves complete product information for the identifiers stored in the user's liked-item array. This allows the Wishlist view to reuse the normal Vivere product-card interface.

The Wishlist serves two purposes. It provides an expected user-facing feature for saving products, but it also creates behavioral data that can be reused by the personalized recommendation system.



4.6 Find Similar Functionality

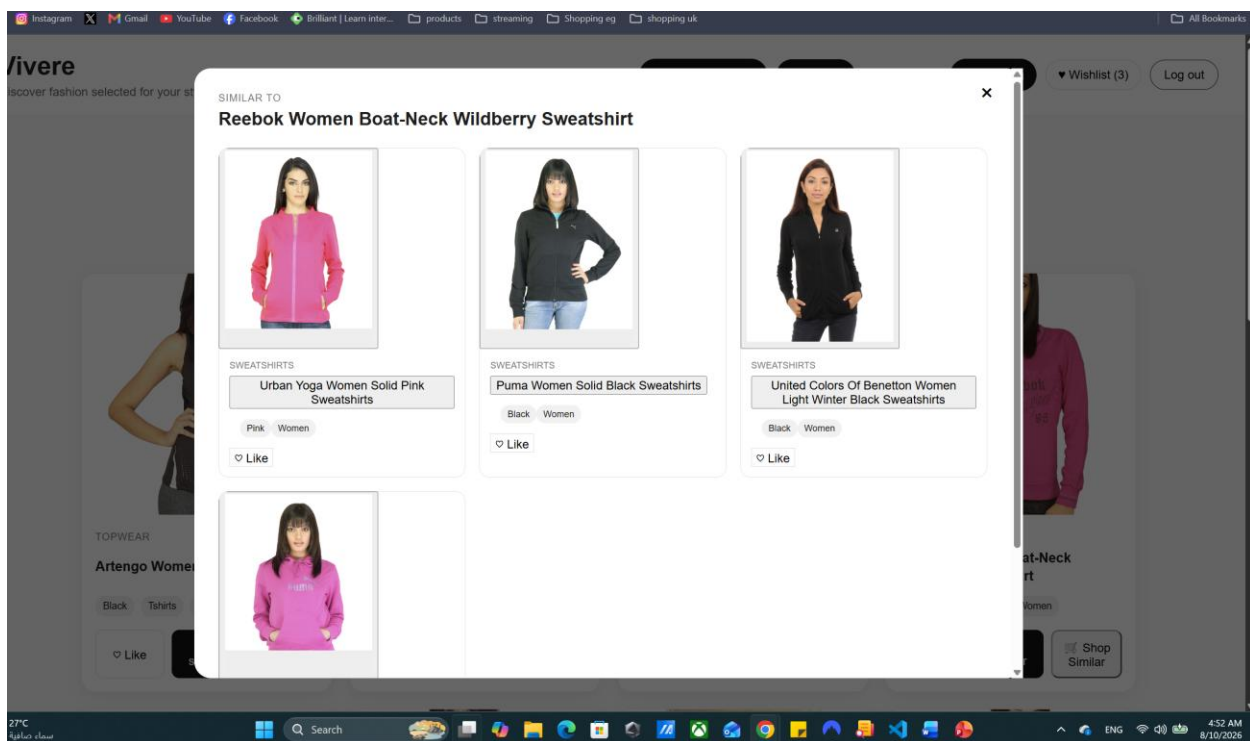
The Find Similar feature allows users to start from an existing product rather than constructing a new search manually.

When a user selects Find Similar, the frontend requests `/api/agent/similar/:id`. The backend first retrieves the selected source product. Candidate items are then located using related fields such as gender, article type and subcategory.

Each candidate receives a similarity score. Matches in article type receive a relatively high contribution because this characteristic strongly describes what the product is. Additional points are given when subcategory, color, usage and season match the source product.

The highest-scoring candidates are retained, and a limited number are returned to the frontend. Some randomization is used within the strongest candidates so that repeatedly requesting similar items does not always return exactly the same products in the same order.

The results are displayed in a modal interface containing images, product attributes and further actions. Users can therefore move between products through similarity rather than returning to the main search interface.



4.7 Personal Stylist

The Personal Stylist represents the explicit-preference recommendation system within Vivere.

Rather than using a fixed list of available options in the React code, the frontend retrieves available genders, colors, usages, seasons and article types from MongoDB using the

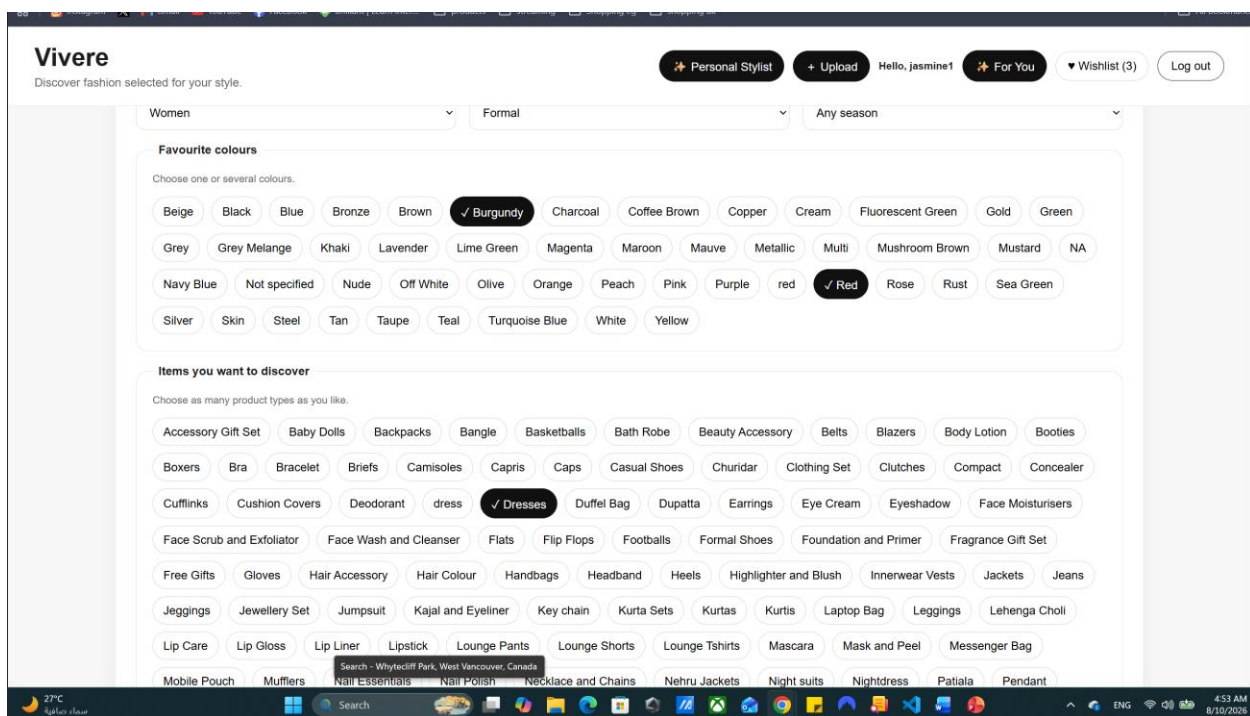
/api/stylist/options route. MongoDB's distinct operation is used to retrieve unique values for each field.

This approach ensures that the Personal Stylist reflects the actual contents of the dataset. If new colors or product types are introduced, they can appear in the interface without requiring the frontend option lists to be rewritten manually.

The user can select a gender, occasion, season, one or multiple colors, and one or multiple article types. The selected preferences are submitted to /api/recommendation.

Candidate products are assigned recommendation scores according to how well they match the selections. Article-type matches receive greater weighting, while color, usage, gender and season contribute additional points. The strongest matches are grouped into a recommendation pool, and a final set of products is returned.

This method produces recommendations that are personalized to the current request while remaining straightforward to explain and debug.



4.8 Recommendations Learned from Likes

A second recommendation mechanism was developed to create personalized results without requiring the user to complete the Personal Stylist form each time.

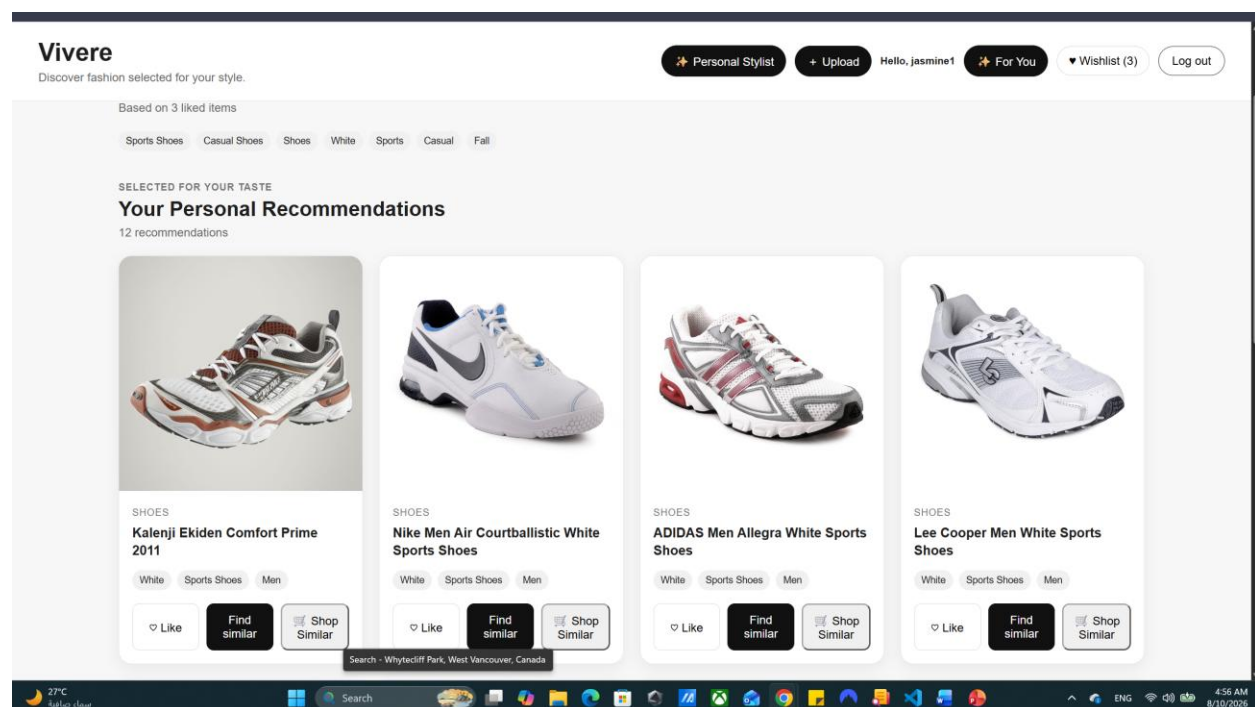
The `/api/users/:userId/recommendations` endpoint first retrieves the user's liked products. The backend then counts the frequency of characteristics appearing within those items, including article type, subcategory, color, usage, season and gender.

These frequency counts form a lightweight user taste profile. For example, if several liked products are blue, blue becomes a stronger signal. If multiple liked products belong to the same article type, that type receives increased importance.

Products already liked by the user are excluded from the candidate pool. Remaining products receive scores according to how many preferred attributes they share with the user's existing likes. Article type and category are weighted more heavily than less specific characteristics such as season.

The resulting products are presented through the **For You** interface. The page also displays characteristics learned from the Wishlist, making the recommendation process more transparent to the user.

This implementation is content-based rather than collaborative. It was appropriate for the project because meaningful recommendations can be produced even when the application has only a small number of registered users.



4.9 Clothing Upload Functionality

User-generated content was introduced through the Upload Clothing feature.

A separate React component provides a form in which users can choose an image and enter product information including name, article type, subcategory, color, gender, main category, season and usage.

The image is submitted to the backend using FormData because the request contains both a file and textual information. The Express backend uses **Multer** to process the multipart upload.

Multer assigns the uploaded image a unique filename and saves the file within the server's uploads directory. Express exposes this directory as static content through the /uploads path, allowing the React frontend to display uploaded images.

Uploaded products are saved into MongoDB using the same primary product schema as the original fashion items. Additional fields were introduced: imageUrl, uploadedBy and isUserUpload

The imageUrl stores the server path to the uploaded file. uploadedBy records the contributing user, while isUserUpload distinguishes uploaded products from the original dataset.

Using the same collection means uploaded products automatically participate in normal search and filtering. This was an important implementation decision because it avoids maintaining a completely separate catalogue for user-generated content.

The React product-image logic checks whether an item contains an imageUrl. If it does, the image is loaded from the backend upload path. Otherwise, the original dataset image structure is used.

The screenshot shows a web browser window with the Vivere website. The header includes the Vivere logo, a tagline 'Discover fashion selected for your style.', and navigation links: 'Personal Stylist', '+ Upload', 'Hello, jasmine1', 'For You', 'Wishlist (3)', and 'Log out'. The main content area is titled 'ADD TO THE VIVERE COLLECTION' and 'Upload Clothing'. Below this is a sub-header 'Upload a fashion item and add information that helps Vivere include it in search and recommendations.' The form itself is a light gray box with a white background, containing several input fields and dropdown menus. The fields are: 'Clothing image *' (with a 'Choose File' button and 'No file chosen' text), 'Product name *' (with an example 'Blue summer dress'), 'Article type *' (with an example 'Dresses, Shirts, Handbags'), 'Subcategory' (with an example 'Topwear, Shoes, Bags'), 'Colour' (with an example 'Blue'), 'Gender' (a dropdown menu with 'Unisex' selected), 'Main category' (a dropdown menu with 'Apparel' selected), 'Season' (a dropdown menu with 'All Seasons' selected), and 'Usage' (a dropdown menu with 'Casual' selected). At the bottom of the form is a large black button labeled 'Upload Clothing Item' and a smaller 'Clear' button. A 'Back to shop' button is located at the top right of the form area.

4.10 Shop Similar

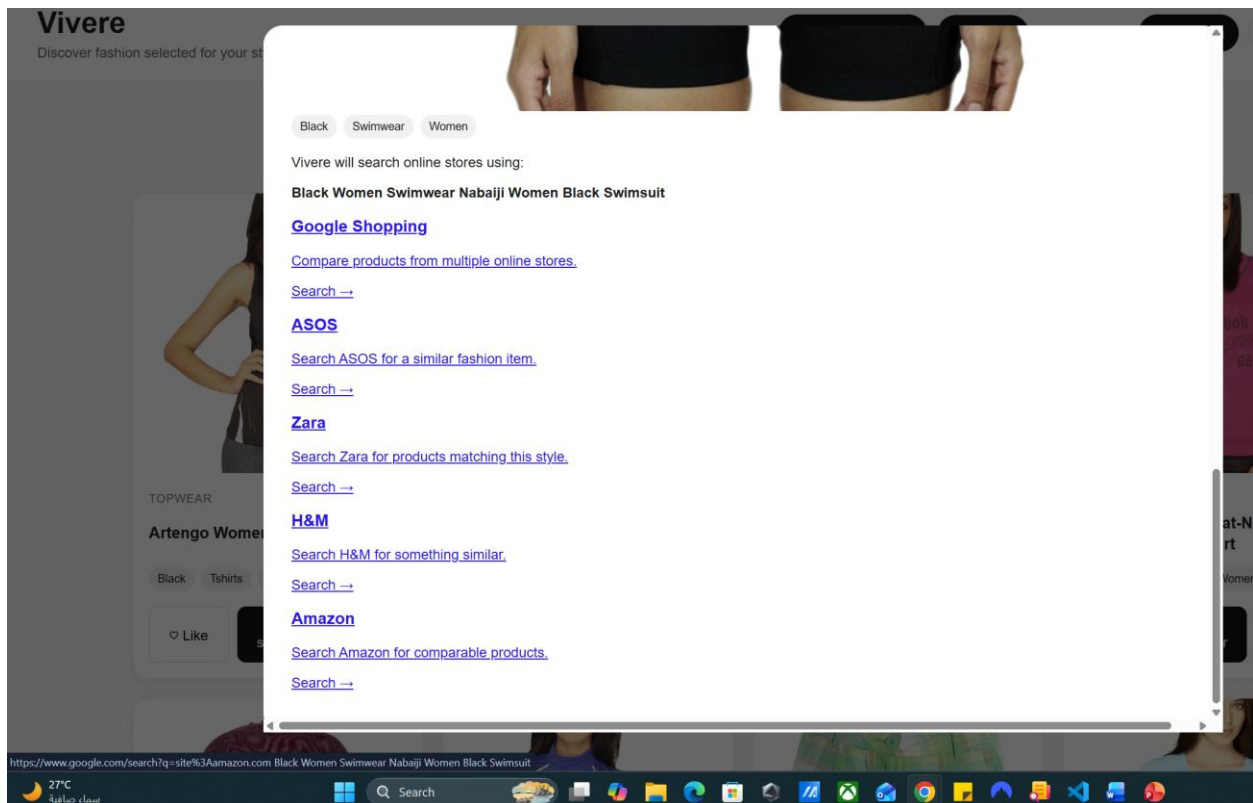
The Shop Similar feature extends Vivere's product discovery process to external online retailers.

A dedicated React modal constructs a search phrase using attributes such as product color, gender, article type and product name. The resulting phrase is URL-encoded and used to create external search links.

The interface provides links for services including Google Shopping and retailer-specific searches such as ASOS, Zara, H&M and Amazon.

This feature should be distinguished from Find Similar. Find Similar searches within the Vivere catalogue using backend similarity scoring, whereas Shop Similar helps the user continue the search outside Vivere.

The current implementation does not retrieve live retailer inventories or prices through commercial APIs. Instead, it creates relevant external searches. This approach avoided dependencies on retailer APIs while still demonstrating integration between Vivere's internal product discovery and external shopping services.



4.11 Error Handling and Development Challenges

Several implementation issues were encountered during development. Database connectivity occasionally failed because MongoDB Atlas network-access rules did not include the current development IP address. This was resolved through configuration of Atlas Network Access settings.

React development also required managing compatibility between the existing Create React App environment and the installed Node.js version. A compatible Node.js version was eventually used to restore reliable frontend compilation.

Uploaded images initially appeared unavailable because product records were being created without the new imageUrl fields. Updating the Mongoose schema and ensuring that the correct upload route was active resolved the issue.

Other development problems included frontend routes being connected incorrectly, duplicate JSX blocks, image-path differences between dataset and uploaded products, and recommendation routes not being served by the expected backend process. These issues were addressed through incremental testing of individual API endpoints before integrating them into the frontend.

This iterative debugging process was useful because it demonstrated the importance of testing each layer independently. Directly opening backend routes in the browser helped determine whether problems originated from MongoDB, Express or React.

4.12 Implementation Summary

The implementation of Vivere transformed the original fashion dataset into an interactive full-stack application. The final system combines browsing, filtering, authentication, Wishlist management, recommendation logic, product similarity, user uploads and external shopping discovery.

The main architectural advantage is that most functionality operates on a shared product representation. Search, Personal Stylist recommendations, liked-item recommendations and user-uploaded content all use the same underlying product attributes. This reduces duplication and makes the different features work together as part of a single fashion-discovery platform.

5. Testing and Evaluation

5.1 Testing Strategy

Testing was carried out throughout the development of Vivere rather than being left until the final stage. Because the application is divided into frontend, backend and database layers, testing was performed at each layer individually and then repeated after integration.

The main strategy combined manual functional testing with direct API testing. Backend routes were frequently tested in the browser or through direct requests before being connected to React. This was particularly useful for identifying whether faults originated in the frontend interface, Express routes or MongoDB queries.

The testing process focused primarily on whether the system met the functional requirements defined earlier in the report. The most important areas included authentication, catalogue retrieval, search, filtering, Wishlist behavior, recommendations, similar-item discovery, uploads and external shopping links.

5.2 Functional Testing

The following tests were performed on the completed application:

Test	Expected Result	Result
Register a new user	User account is created and returned by the backend	Passed

Test	Expected Result	Result
Attempt duplicate registration	System prevents duplicate username or email	Passed
Log in with valid details	User is authenticated and personalized controls appear	Passed
Browse catalogue	Fashion products load in paginated groups	Passed
Search by keyword	Matching products are returned	Passed
Filter by gender	Only products with the selected gender are displayed	Passed
Filter by color	Matching color results are returned	Passed
Filter by season or usage	Product list updates correctly	Passed
Open product details	Full product metadata is displayed	Passed
Like an item	Product is added to the user's liked items	Passed
Unlike an item	Product is removed from liked items	Passed
Open wishlist	Saved products are displayed	Passed
Use Personal Stylist	Recommendations reflect selected preferences	Passed
Open For You recommendations	Results are generated from previously liked products	Passed
Find Similar	Related products from the Vivere catalogue are returned	Passed
Upload an item	Image and metadata are stored successfully	Passed
Search for uploaded item	Uploaded product appears in catalogue search	Passed
Display uploaded image	Correct uploaded image is shown	Passed
Shop Similar	External shopping searches open for the selected product	Passed

The results demonstrate that the major functional requirements were successfully implemented.

5.3 Authentication Testing

Authentication was tested using both valid and invalid user input.

Registration was tested by creating a new username, email address and password. The server successfully created the corresponding MongoDB document and returned the account information to the frontend.

Duplicate registration attempts were then performed using an existing username or email address. The backend correctly rejected these requests instead of creating duplicate user records.

Login was tested with both usernames and email addresses because the login endpoint supports either option. Correct credentials returned the appropriate user information, while invalid combinations produced an authentication error.

The frontend was also refreshed after successful login to determine whether the user remained represented within the interface. Because login information is stored in browser local storage, the application was able to reconstruct the logged-in state after a page refresh.

Although these behaviors satisfy the prototype requirements, the authentication implementation has an important security limitation. Passwords are currently stored and compared directly rather than using cryptographic password hashing. For a production system, a library such as bcrypt would be required together with stronger session or token-based authentication.

5.4 Search, Filtering and Pagination Testing

Search functionality was tested using product names, colors and article types. The backend successfully returned products when the search term matched different supported metadata fields.

Filtering was tested independently and in combination. For example, selecting a gender and color reduced the catalogue to products satisfying both conditions. Changing filters also reset pagination to the first page, preventing the interface from remaining on a page number that might not exist within the filtered result set.

Pagination was tested by navigating between product pages using the Previous and Next buttons. Boundary conditions were also checked. The Previous button was disabled on page one, while the Next button was disabled when the final page had been reached.

These tests confirmed that the system could operate on the large dataset without attempting to render the entire collection simultaneously.

5.5 Wishlist Testing

Wishlist functionality was tested by adding and removing several products.

When the user selected Like, the correct product identifier was added to the user's likedItems array in MongoDB. The frontend immediately updated the button state and Wishlist count.

Selecting an already-liked product again successfully removed it from the array. The Wishlist view was then opened to confirm that the corresponding product disappeared from the displayed list.

The test also confirmed that Wishlist information was available after page refresh because the backend remained the persistent source of user data.

An additional benefit of this test was confirming that Wishlist interactions could subsequently be reused by the recommendation system.

5.6 Personal Stylist Testing

The Personal Stylist was tested with multiple combinations of explicit preferences.

For example, selections were made for gender, color, season, usage and product type. The backend logged the submitted preference object and returned products with matching characteristics.

A significant issue identified during early testing was that different preference combinations initially produced highly repetitive recommendations. This was investigated by checking the request data being sent to the backend and adjusting the recommendation logic so that selected colors and product types influenced the score correctly.

The stylist option lists were later changed so that they were generated from the MongoDB dataset rather than hard-coded in React. This was tested by opening the options endpoint directly and confirming that unique genders, colors, usages, seasons and article types were returned.

The final implementation produced visibly different recommendations when preference combinations changed, demonstrating that the recommendation engine was responding to the user's selections rather than returning a fixed result set.

5.7 For You Recommendation Testing

The personalized For You feature was tested using a user account containing liked products.

The endpoint first returned a taste profile identifying characteristics found within those products. During testing, attributes such as favorite article types, subcategories, colours, usages and seasons were visible in the returned JSON response.

The endpoint then returned a collection of products that were not already present in the user's Wishlist but shared characteristics with the user's preferences.

This test demonstrated an important difference between the two recommendation systems. The Personal Stylist depends on explicit selections made during the current interaction, while For You can generate results from previously recorded behavior.

The recommendation logic was also tested with a user who had no liked items. Instead of returning an error, the application displayed a message informing the user that products should be liked before personalized recommendations could be created.

5.8 Similar Product Testing

The Find Similar feature was tested using products from several categories.

Selecting a product caused the backend to retrieve its metadata and identify candidates with related article types or subcategories. Similarity scores were then calculated using fields such as article type, subcategory, color, usage and season.

The returned products were inspected manually to determine whether they were meaningfully related to the source item. Results generally contained products belonging to the same or closely related category, confirming that the scoring logic was functioning as intended.

The use of randomization among the strongest matches also produced some variety between repeated requests while maintaining general relevance.

5.9 Upload Testing

The upload feature required testing both file handling and database integration.

A fashion image was selected through the React upload interface together with metadata including product name, article type, colour, gender, season and usage.

The file was transmitted to the backend using multipart form data. Multer successfully processed the upload and stored the image inside the server upload directory.

The corresponding MongoDB record was then inspected. The record contained the entered product metadata together with an imageUrl and information identifying it as a user-uploaded item.

An early version of the feature revealed an implementation problem in which uploaded products appeared in search results but displayed the Image Unavailable placeholder. Inspection of the database response showed that the uploaded records did not contain imageUrl. The Mongoose schema and upload route were corrected and the backend restarted.

After this correction, a newly uploaded product successfully displayed its image in the catalogue.

This testing process confirmed the complete upload flow:

image selection → file upload → server storage → MongoDB record → search → frontend image display

5.10 Shop Similar Testing

Shop Similar was tested using several products with different colors and article types.

The feature constructed a text query from the selected product's metadata and displayed links for external shopping services such as Google Shopping, ASOS, Zara, H&M and Amazon.

Each link was tested to confirm that it opened in a separate browser tab and carried a relevant search phrase.

The feature worked as intended, although it is important to recognize its technical limitation. Vivere does not directly query live retailer inventories. Instead, it generates external search links. Therefore, the availability and relevance of results ultimately depend on the external search provider.

5.11 Usability Evaluation

The interface was evaluated manually by completing common user journeys from beginning to end.

The main navigation places high-value functionality close to the top of the interface. Registered users can access Personal Stylist, Upload, For You, Wishlist and logout functionality from the header. This reduces the number of actions required to reach core features.

Product cards use consistent actions across different views, making interactions predictable. The same products can be liked or examined whether they appear in the main catalogue, recommendations or similar-item results.

The use of loading messages and empty-state messages improves feedback during asynchronous operations. For example, the application displays messages while recommendations or Wishlist content are loading and provides explanations when no products are available.

Some interface areas remain visually simple because development priority was given to functional requirements. Additional responsive design testing and accessibility improvements would therefore be valuable before a commercial release.

5.12 Performance Evaluation

The large dataset created a potential performance issue because approximately 44,000 items are stored within the catalogue.

Pagination significantly reduced this problem. The frontend normally requests twelve products for a page instead of retrieving the entire collection.

Database filtering is also performed by the backend before results are returned. Therefore, selecting a color or gender does not require React to download thousands of irrelevant records and filter them locally.

The recommendation endpoints perform more substantial processing because candidate items must be examined and scored. For the scale of the prototype, response times remained acceptable during testing. However, a production application with substantially more products or users would benefit from additional optimization, indexing and potentially pre-computed recommendation data.

5.13 Reliability and Error Handling

Error-handling behavior was tested throughout development.

If a product image fails to load, a placeholder image is displayed. Invalid item identifiers produce appropriate backend responses rather than server crashes. Failed API requests are caught by the React interface and result in status messages or empty states.

Database connectivity represented one of the most significant external reliability issues during development. MongoDB Atlas rejected connections when network-access configuration did not permit the current IP address. Updating Atlas Network Access resolved the problem.

This demonstrates that cloud-hosted dependencies introduce configuration requirements outside the application source code itself.

5.14 Evaluation Against Project Objectives

The completed system can be evaluated against the objectives established in Section 1.

The objective of creating an accessible fashion catalogue was achieved through the main React browsing interface. Search, filtering and pagination were successfully implemented.

User accounts and persistent Wishlist functionality were also achieved.

The project successfully developed two different recommendation mechanisms: explicit preference recommendations through Personal Stylist and implicit preference recommendations through liked-item analysis.

Similar-item discovery was implemented within the internal catalogue, while Shop Similar extended discovery to external shopping searches.

The objective of allowing user-generated content was achieved through the clothing upload feature, and uploaded products were successfully incorporated into normal search functionality.

The project therefore satisfied the majority of its planned functional objectives.

5.15 Limitations

Despite meeting the main requirements, several limitations remain.

The recommendation system is based primarily on structured metadata and manually selected weighting values. It does not use deep-learning image embeddings, collaborative filtering across large user populations or trained ranking models.

The similarity system therefore represents semantic similarity based on catalogue attributes rather than visual similarity between images.

The Shop Similar feature does not retrieve live retailer product data, prices or availability through retailer APIs.

Authentication is suitable only for demonstration purposes because passwords should be hashed and more secure session handling should be introduced.

Uploaded images are stored on the local backend server. This works during local development but would not be suitable for a scalable distributed deployment. Production deployment would require dedicated object storage or an image-hosting service.

The current application also does not provide administrative moderation for user-uploaded products. A public system would require content validation, ownership controls and potentially reporting functionality.

5.16 Testing and Evaluation Summary

Testing demonstrated that the major Vivere features operate successfully as an integrated application. Search, filtering, authentication, Wishlist management, recommendation functionality, product similarity, uploads and external discovery all produced the expected behavior.

The testing process also identified several implementation issues, particularly around database connectivity, recommendation variability and uploaded-image handling. Resolving these issues improved the reliability of the completed prototype.

Overall, Vivere achieved its objective of creating a personalized full-stack fashion discovery system while also identifying clear opportunities for further technical development.

6. Conclusion and Future Work

6.1 Conclusion

The aim of the Vivere project was to design and develop a personalized fashion discovery web application that combines conventional product browsing with recommendation and discovery functionality. The completed system demonstrates how a large fashion dataset can be transformed from a static collection into an interactive platform that responds to individual user preferences and behavior.

Vivere was developed as a full-stack application using React for the frontend, Node.js and Express for the backend, and MongoDB Atlas for persistent data storage. The application successfully integrates a fashion dataset containing approximately 44,000 products while maintaining manageable performance through server-side filtering and pagination.

The fundamental catalogue functionality allows users to browse products, perform keyword searches and filter results according to characteristics such as gender, color, season, category and usage. These functions provide users with direct control when they already understand the products they wish to find.

The project also explored more personalized approaches to fashion discovery. The Personal Stylist enables users to explicitly describe their current requirements and receive products matching those preferences. In comparison, the For You recommendation system uses characteristics derived from previously liked products to create a lightweight representation of the user's fashion interests. The combination of these approaches

demonstrates how both explicit and implicit preference information can contribute to personalization.

The Find Similar functionality provides another method of navigating the catalogue by identifying products that share characteristics with an existing item. Shop Similar extends this concept beyond the internal dataset by constructing external searches that allow users to continue looking for comparable products through online shopping services.

User-generated content was another significant component of the final application. Registered users can upload clothing images together with descriptive metadata. The backend processes the uploaded file and stores the corresponding product information within MongoDB. Importantly, uploaded products are integrated into the existing catalogue rather than being treated as an isolated feature. They can therefore appear in searches alongside products from the original dataset.

Development of the project also demonstrated the importance of iterative testing within full-stack applications. Several challenges occurred during implementation, including MongoDB connectivity, frontend compilation, recommendation behavior and image-upload handling. Debugging these issues required individual testing of the frontend, backend API and database. This process ultimately resulted in a more reliable integrated system.

Overall, the completed prototype meets the principal objectives established at the beginning of the project. Vivere demonstrates that search, recommendation, user interaction and external product discovery can be combined within a single fashion-focused application.

6.2 Future Work

Although the current system provides the required functionality, there are several opportunities for further development.

One significant improvement would be the introduction of **visual similarity search**. The existing Find Similar system compares structured product metadata such as article type, category and color. A future version could use computer-vision models to generate embeddings from clothing images. Similarity between these embeddings could then allow users to upload an image and discover visually comparable products even when detailed metadata is unavailable.

The recommendation system could also be extended through machine-learning techniques. The current approach is content-based and uses manually weighted product characteristics. With a substantially larger user population, collaborative filtering could

identify relationships between users with similar preferences. A hybrid recommendation system combining collaborative information, product metadata and visual characteristics could potentially produce more sophisticated recommendations.

Another improvement would involve the Shop Similar functionality. The current implementation generates relevant external searches. Integration with authorized retailer or product-search APIs could instead allow Vivere to retrieve current product names, images, prices and purchase links directly. This would create a more complete connection between product inspiration and purchasing.

User-uploaded images could also be moved from local server storage to a dedicated cloud image-storage service. This would improve scalability and make deployment across multiple servers considerably easier. Image optimization could additionally reduce loading times and bandwidth requirements.

Security would require substantial improvement before public deployment. Passwords should be hashed using an appropriate cryptographic password-hashing algorithm, and authentication should use secure sessions or tokens. Additional input validation, upload restrictions, rate limiting and server-side security controls would also be required.

A public version of Vivere would additionally benefit from an administration and moderation system. Because users can contribute content, mechanisms would be required for removing inappropriate uploads, reporting content and managing user accounts.

Finally, further work could improve accessibility and responsive design. Testing the interface across a larger range of screen sizes and devices would ensure a more consistent experience. Accessibility evaluation could also be performed against WCAG recommendations, including keyboard navigation, semantic markup, alternative text and sufficient visual contrast.

6.3 Final Reflection

Vivere demonstrates the practical application of concepts from full-stack web development, database management, recommendation systems and user-centered software design. The project progressed beyond displaying a fashion dataset by introducing persistent user accounts, behavioral information, personalized recommendations, user-generated content and multiple methods of product discovery.

The resulting application provides a functional foundation that could be extended considerably in the future. More importantly, the project demonstrates how different technologies and software-engineering concepts can be integrated to solve a clear

problem: helping users navigate a large fashion catalogue and discover products that are more relevant to their individual interests.

References:

1. Bobadilla, J., Ortega, F., Hernando, A. and Gutiérrez, A. (2013). Recommender systems survey. *Knowledge-Based Systems*, 46, pp.109–132.
2. Deldjoo, Y. et al. (2022). A Review of Modern Fashion Recommender Systems.
3. Ding, Y., Lai, Z., Mok, P.Y. and Chua, T.-S. (2023). Computational Technologies for Fashion Recommendation: A Survey.
4. McKinsey & Company and The Business of Fashion (2024). The State of Fashion 2025.